

Engineering-First Deep Understanding (Part 2)

4. Distributed Systems Failure Modes: What Breaks in Production

From Distributed Systems Engineer Perspective

Core Reality: In distributed systems, partial failure is the default. "Up" and "down" are abstractions that rarely match reality.

Production Scenario 8: The Partial Failure Nobody Noticed

Setting: Microservices with 10 instances each

Incident:

```
Service A: 10 instances running
Service B: 10 instances running
Network: 1 instance of A cannot reach 1 instance of B
```

Observable Symptoms:

```
Service A metrics:
- Success rate: 90%
- Error rate: 10%
- All instances report "healthy"

Service B metrics:
- Success rate: 100%
- No errors detected
```

What's Happening:

```
A-instance-1 → B-instance-1 ✓  
A-instance-1 → B-instance-2 ✓  
...  
A-instance-5 → B-instance-3 ✗ (network partition)
```

Why It's Hard to Detect:

- Load balancer sees B-instance-3 as healthy (responds to health checks)
- Only A-instance-5's requests to B-instance-3 fail
- 90% of traffic succeeds (other instance pairs work)
- B's metrics show no errors (from its perspective, nothing is wrong)

Correct Detection:

```
# Alert on per-instance error rate  
rate(errors{instance="A-5"}[5m]) > 0.1  
  
# Cross-reference with destination  
rate(errors{src="A-5", dst="B-3"}[5m])
```

Production Scenario 9: The Cascade That Started Small

Timeline:

```
10:00: Service C response time: 100ms → 200ms (+100ms)  
      Service B timeout: 5000ms (safe)  
      No impact  
  
10:15: Service C response time: 200ms → 500ms (+300ms)  
      Still within B's timeout  
      No errors  
  
10:30: Service C response time: 500ms → 2000ms  
      B's thread pool starts filling  
      B still responding normally  
  
10:45: Service C response time: 2000ms → 4000ms  
      B's thread pool 90% full  
      B latency increases: 100ms → 500ms  
  
11:00: Service C response time spikes to 8000ms  
      B's threads exhausted waiting for C  
      B stops responding
```

```
B's upstream (Service A) times out
A cascades failure upstream
```

```
11:15: Total outage across 5 services
```

Why Traditional Monitoring Missed It:

Traditional alerts:

- "Service is down" → Not triggered (services responded until 11:00)
- "Error rate > 5%" → Not triggered (no errors, just slow)
- "CPU > 80%" → Not triggered (CPU was low, threads were blocked)

What Should Have Been Monitored:

Saturation metrics:

- Thread pool utilization: Would show 90% at 10:45
- Request queue depth: Would show increase at 10:30
- Downstream latency: Would show C degradation at 10:00

Alert that would have caught it:

```
"Service B thread pool utilization > 70% for 10 minutes"
```

From Platform Engineer Perspective

Resource Exhaustion Hierarchy (Things That Run Out):

1. File Descriptors (per process limit)
ulimit -n → typically 1024
Each: socket, open file, pipe
Exhaustion symptom: "Too many open files"
2. TCP Ports (per IP)
Ephemeral port range: typically 32768-60999
~28,000 available
Exhaustion symptom: "Cannot assign requested address"
3. Connection Pool (application limit)
Database: 10-100 connections typical
HTTP client: 50-200 connections typical
Exhaustion symptom: Requests queue/timeout
4. Thread Pool (application limit)

```
Tomcat default: 200 threads
Exhaustion symptom: Thread creation failure
```

5. Memory (OS limit)
Can be physical or cgroup limit
Exhaustion symptom: OOM kill
6. CPU (relative, not exhaustible)
Can be fully utilized but doesn't "run out"
Symptom: High scheduling latency

Production Scenario 10: Thundering Herd via Cache Stampede

Setup:

```
Service: Product API
Traffic: 10,000 req/s
Cache: Redis, TTL = 5 minutes
Database: MySQL, capacity = 500 req/s
```

Incident:

```
14:00:00 - Cache entry for popular product expires
14:00:00 - Next 10,000 requests (1 second of traffic) miss cache
14:00:00 - All 10,000 requests hit database simultaneously
14:00:01 - Database overloaded (can handle 500 req/s, received 10,000)
14:00:02 - Database connection pool exhausted
14:00:03 - All subsequent requests fail (can't connect to DB)
14:00:05 - Database recovered, but now 50,000 requests queued
14:00:10 - Death spiral continues for 5 minutes
```

Why Retries Made It Worse:

```
# Client code
def get_product(id):
    for attempt in range(3):
        try:
            return api.get(f"/product/{id}")
        except:
            time.sleep(1)
    raise Exception("Failed after 3 retries")
```

```
# Result: Each failure triggers 3 total requests
# 10,000 initial requests → 30,000 requests to DB
# Amplification factor: 3x
```

Correct Solutions:

Solution 1: Request Coalescing

```
# Only one request per key goes to DB, others wait
from threading import Lock

_in_flight = {}
_locks = {}

def get_product(id):
    if id in _in_flight:
        return _in_flight[id].wait() # Wait for in-flight request

    lock = _locks.setdefault(id, Lock())
    with lock:
        if id in cache:
            return cache[id]

        _in_flight[id] = Future()
        result = db.get(id)
        cache.set(id, result, ttl=300)
        _in_flight[id].set_result(result)
        del _in_flight[id]
        return result
```

Solution 2: Probabilistic Early Recomputation

```
import random

def get_product(id):
    value, ttl_remaining = cache.get_with_ttl(id)

    # Recompute early with probability inversely proportional to TTL
    if random.random() < (1.0 / ttl_remaining):
        value = db.get(id)
        cache.set(id, value, ttl=300)

    return value
```

Solution 3: Stale-While-Revalidate

```
def get_product(id):
    value = cache.get(id)

    if value and value.is_stale():
        # Return stale value immediately
        # Asynchronously refresh in background
        asyncio.create_task(refresh_cache(id))
        return value.data

    return value.data if value else db.get(id)
```

From Hiring Manager Perspective

Interview Question 8:

"You have Service A with 10 instances calling Service B with 10 instances. 10% of requests are failing. How do you debug this?"

Correct Answer:

- Check per-instance error rates (not aggregate)
- Check error rate by source-destination pair
- Look for network partitions between specific instances
- Don't assume "Service B is down" from aggregate metrics

Red Flags:

- Only checks aggregate metrics
- Assumes all instances are identical
- Doesn't consider partial failures

Interview Question 9:

"Your service uses a cache with 5-minute TTL. Cache hit rate is 99%. Traffic is 1000 req/s. Database can handle 100 req/s. What happens when cache expires?"

Correct Answer:

- 1000 req/s suddenly hit database (cache miss)
- Database overloads (capacity is 100 req/s)
- Need request coalescing or stale-while-revalidate
- Calculate: $1000 / 100 = 10x$ over capacity

Red Flags:

- "1% of 1000 = 10 req/s to database" (doesn't understand stampede)
- Doesn't recognize the problem
- Suggests "just scale database"

Lab Exercises

Easy: Partial Network Partition

Setup: Docker Compose with Services A (3 instances) and B (3 instances)

Exercise:

1. Use `iptables` to block traffic from A-instance-2 to B-instance-2 only
2. Measure aggregate success rate (should be ~89%, not 0% or 100%)
3. Identify which instance is failing using per-instance metrics
4. Detect using PromQL query

Success Criteria:

- Correctly identify affected instance pair
- Alert fires on per-instance metric
- Aggregate metric shows partial failure

Moderate: Cascade Simulation

Setup: Services A → B → C, each with thread pool (size=10)

Exercise:

1. Inject latency in C: 100ms → 5000ms (gradually over 10 minutes)
2. Measure:
 - When does C's thread pool saturate?
 - When does B's thread pool saturate?
 - When does A start returning errors?
3. Calculate: Cascade propagation time

Success Criteria:

- Predict saturation points using math ($\text{pool_size} \times \text{latency}$)
- Measure matches prediction
- Identify: At what C latency should B have opened circuit breaker?

Hard: Thundering Herd Mitigation

Setup: API with Redis cache (TTL=60s), Database (capacity=100 req/s), Load test (1000 req/s)

Exercise:

1. Baseline: Measure DB load when cache expires (should spike to 1000 req/s)
2. Implement request coalescing
3. Re-test: Measure DB load when cache expires (should stay at ~1 req/s)
4. Measure: What's the P99 latency with vs without coalescing?

Success Criteria:

- Request coalescing limits DB load to 1 req/s
- P99 latency improves (no DB overload)
- Prove: All 1000 concurrent requests get same cached result

Expert-Level Diagnostic Questions

Question 8:

Service A has 100 instances. Service B has 10 instances. Each A instance opens 5 persistent connections to B. How many connections does each B instance handle? What's the failure mode when B scales to 20 instances?

Expected Answer:

- Total connections: $100 \times 5 = 500$
- Per B instance: $500 / 10 = 50$ connections
- After scaling: $500 / 20 = 25$ connections per instance
- Failure mode: Connection pool in B might be sized for 50, not 25
- If using connection pooling, may need to rebalance

Question 9:

You implement retry logic with exponential backoff: 1s, 2s, 4s. Max 3 retries. Your service handles 1000 req/s with 1% error rate. How much extra load do retries add?

Expected Answer:

- Errors: $1\% \times 1000 = 10$ req/s fail
- Each failed request retries 3 times
- Extra load: $10 \times 3 = 30$ req/s
- Total load: $1000 + 30 = 1030$ req/s (3% increase)
- At 10% error rate: $1000 + (100 \times 3) = 1300$ req/s (30% increase!)
- This is why error rates cascade

5. Prometheus + Grafana: Production Observability

From Observability Architect Perspective

Prometheus is not just a metrics database. It's a pull-based discovery and scraping system.

Production Scenario 11: The Metric That Killed Prometheus

Incident:

```
15:00: Deployed new feature with user-level metrics
15:30: Grafana dashboards loading slowly
16:00: Prometheus OOM (Out of Memory)
16:15: Complete observability blackout during incident
```

Root Cause:

```
# Bad: High cardinality label
user_request_count.labels(user_id=user_id).inc()

# With 1M active users
# Each with metrics for: requests, errors, latency
# = 1M × 3 = 3M time series

# Prometheus memory: ~1-3KB per time series
# = 3M × 2KB = 6GB just for this metric
```

Why It's Bad:

- Labels create new time series
- High cardinality labels (user_id, IP, session_id) explode series count
- Prometheus must keep all series in memory
- Query time increases exponentially with series count

Correct Approach:

```
# Good: Log user_id, aggregate in metrics
request_count.labels(endpoint="/checkout", status="200").inc()

# For user-specific analysis: Use logs or tracing
# Logs: "user_id=12345 requested /checkout"
# Query logs for specific user, use metrics for aggregates
```

Cardinality Rules:

- Labels: Use for dimensions you want to aggregate by (endpoint, status, method)
- Logs: Use for high-cardinality data (user_id, request_id, session_id)
- Traces: Use for request flow across services

Production Scenario 12: The Missing Scrape Target

Incident:

```
User Report: "Checkout is down"
Dashboard: Shows no data for Checkout service
Reality: Checkout is running fine
Problem: Prometheus isn't scraping it
```

Root Cause Investigation:

```
# Check Prometheus targets
curl http://prometheus:9090/api/v1/targets | jq '.data.activeTargets[] | select(.labels.'

# Result: No targets found

# Check service discovery
kubectl get pods -l app=checkout
# Result: 3 pods running

# Check service annotations
kubectl get service checkout -o yaml
# Missing: prometheus.io/scrape: "true"
```

Fix:

```
# Add Prometheus annotations to Kubernetes service
apiVersion: v1
kind: Service
metadata:
  name: checkout
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
    prometheus.io/path: "/metrics"
```

Lesson: Metrics only exist if Prometheus can find and scrape them

From SRE Perspective

Essential PromQL Patterns:

1. Rate Calculation (Requests per Second)

```
# Wrong: Just the counter value
http_requests_total

# Right: Rate over time window
rate(http_requests_total[5m])

# Why: Counters always increase, rate shows change per second
```

2. Error Rate (Percentage)

```
# Wrong: Absolute error count
rate(http_requests_total{status="500"}[5m])

# Right: Error percentage
rate(http_requests_total{status=~"5.."}[5m])
/
rate(http_requests_total[5m])
* 100

# Even better: Use recording rule for efficiency
```

3. Latency Percentiles

```
# Wrong: Average latency (hides tail)
avg(http_request_duration_seconds)

# Right: P99 latency
histogram_quantile(0.99,
  rate(http_request_duration_seconds_bucket[5m])
)

# Why: P99 catches outliers that average misses
```

4. Saturation Metrics

```
# Thread pool utilization
thread_pool_active / thread_pool_max

# Connection pool utilization
connection_pool_active / connection_pool_max

# Memory utilization
```

```
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/
node_memory_MemTotal_bytes
```

5. Apdex Score (User Satisfaction)

```
# Apdex = (satisfied + tolerating/2) / total
# Satisfied: < 100ms, Tolerating: 100-500ms, Frustrated: > 500ms

(
  sum(rate(http_request_duration_seconds_bucket{le="0.1"}[5m]))
  +
  sum(rate(http_request_duration_seconds_bucket{le="0.5"}[5m])) / 2
)
/
sum(rate(http_request_duration_seconds_count[5m]))
```

Production Scenario 13: The Alert That Fired Too Late

Setup:

```
# Bad alert rule
groups:
  - name: api_alerts
    rules:
      - alert: HighErrorRate
        expr: rate(http_requests_total{status="500"}[5m]) > 10
        for: 5m
```

Incident Timeline:

```
14:00: Error rate spikes to 50 req/s
14:00: Alert condition met
14:05: Alert fires (after 5-minute "for" duration)
14:05: On-call paged
14:10: On-call acknowledges, starts investigating
14:20: Service recovered (after 20 minutes of errors)
```

Why This is Bad:

- 5-minute delay before alert fires

- Users already impacted for 5 minutes
- MTTR starts late

Better Alert Rule:

```
groups:
  - name: api_alerts
    rules:
      - alert: HighErrorRate
        expr: |
          (
            rate(http_requests_total{status=~"5.."}[5m])
            /
            rate(http_requests_total[5m])
          ) > 0.05
        for: 2m # Shorter: reduce false positives but fire faster
        annotations:
          summary: "Error rate above 5% for 2 minutes"
          dashboard: "https://grafana/dashboard/api"
```

Alert Tuning Tradeoffs:

```
Short "for" duration:
+ Faster detection
- More false positives

Long "for" duration:
+ Fewer false positives
- Slower detection

Sweet spot: 2-5 minutes for most alerts
```

From Hiring Manager Perspective

Interview Question 10:

"You add a metric `user_requests_total{user_id="..."}` and Prometheus OOMs. Why? How do you fix it?"

Correct Answer:

- High cardinality: `user_id` creates millions of time series
- Each time series consumes memory
- Fix: Remove `user_id` label, put in logs instead
- Metrics for aggregates, logs for high-cardinality data

Red Flags:

- "Just add more memory to Prometheus"
- Doesn't understand cardinality
- Suggests keeping user_id label

Interview Question 11:

"Your service is running but no metrics appear in Prometheus. How do you debug?"

Correct Answer:

1. Check Prometheus targets page (is service listed?)
2. Check service discovery (are pods/services labeled correctly?)
3. Check /metrics endpoint (is it accessible?)
4. Check firewall/network (can Prometheus reach service?)

Red Flags:

- Only checks the service itself
- Doesn't understand Prometheus scraping model
- Blames Prometheus without verification

Lab Exercises

Easy: Metrics Implementation

Setup: Simple HTTP API (no metrics yet)

Exercise:

1. Add Prometheus client library
2. Implement:
 - Counter: `http_requests_total{method, endpoint, status}`
 - Histogram: `http_request_duration_seconds`
3. Expose `/metrics` endpoint
4. Configure Prometheus to scrape
5. Query metrics in Prometheus UI

Success Criteria:

- Metrics visible in Prometheus
- Can calculate request rate and P99 latency
- Proper label cardinality (no high-cardinality labels)

Moderate: Recording Rules and Alerts

Setup: Prometheus scraping multiple services

Exercise:

1. Create recording rule for error rate:

```
- record: job:http_request_errors:rate5m
  expr: |
    rate(http_requests_total{status=~"5.."}[5m])
    /
    rate(http_requests_total[5m])
```

1. Create alert using recording rule:

```
- alert: HighErrorRate
  expr: job:http_request_errors:rate5m > 0.05
  for: 2m
```

1. Test: Inject errors, verify alert fires

Success Criteria:

- Recording rule computed correctly
- Alert fires within 2 minutes of error injection
- Alert includes proper annotations (dashboard link, runbook)

Hard: High Availability Prometheus

Setup: Single Prometheus instance

Exercise:

1. Deploy second Prometheus instance (same config)
2. Both scrape same targets
3. Configure Grafana to query both (Prometheus datasource with failover)
4. Test: Kill one Prometheus, verify dashboards still work
5. Measure: How long does failover take?

Success Criteria:

- No data loss when one Prometheus dies
- Dashboards automatically failover
- Understand tradeoff: duplicate storage

Expert-Level Diagnostic Questions

Question 10:

You have 1000 services, each exposing 100 metrics. Prometheus scrapes every 15 seconds. How many samples per second is Prometheus ingesting?

Expected Answer:

- Total metrics: $1000 \times 100 = 100,000$
- Scrape interval: 15s
- Samples per second: $100,000 / 15 = 6,666$ samples/s
- This affects Prometheus storage and query performance

Question 11:

You want P99 latency alerts. You use histogram with buckets: [0.1, 0.5, 1, 5, 10]. Your P99 is actually 2.5s. What does `histogram_quantile(0.99, ...)` return?

Expected Answer:

- histogram_quantile interpolates between buckets
 - Buckets: 1s and 5s
 - P99 between these buckets
 - Returns: ~2.5s (interpolated between 1 and 5)
 - But: Less accurate than having a 2s or 3s bucket
 - Lesson: Choose bucket boundaries based on SLO thresholds
-

6. Retry Logic & Circuit Breakers: Implementation Details

From Distributed Systems Engineer Perspective

Retries are dangerous. They can turn a small problem into a cascading failure.

Production Scenario 14: The Retry Storm Death Spiral

Setup:

```
Service A: 1000 req/s → Service B
Service B: Capacity 1000 req/s
Service B: 5% error rate (50 req/s fail)
```

Without Retries:


```
Success: 950 req/s
Failures: 50 req/s
Load on B: 1000 req/s
System stable
```

With Naive Retries (retry 3x):

```
def call_service_b():
    for i in range(3):
        try:
            return service_b.call()
        except:
            continue
```

Result:

```
Initial failures: 50 req/s
Retries:  $50 \times 3 = 150$  req/s
Total load:  $1000 + 150 = 1150$  req/s
B now overloaded (capacity: 1000)
Error rate increases: 5%  $\rightarrow$  15%
More retries triggered
Load:  $1000 + (150 \times 3) = 1450$  req/s
Death spiral continues
```

Correct Retry Logic:

```
import random
import time

def call_with_retry(func, max_attempts=3):
    for attempt in range(max_attempts):
        try:
            return func()
        except RetryableError as e:
            if attempt == max_attempts - 1:
                raise

            # Exponential backoff with jitter
            base_delay = 2 ** attempt
            jitter = random.uniform(0, base_delay * 0.1)
            delay = base_delay + jitter
```

```
        time.sleep(delay)
    except NonRetryableError as e:
        # Don't retry 4xx errors
        raise
```

Key Retry Principles:

1. Only Retry Transient Errors

```
RETRYABLE_ERRORS = [
    TimeoutError,
    ConnectionError,
    503 # Service Unavailable
]

NON_RETRYABLE_ERRORS = [
    400, # Bad Request (client error)
    401, # Unauthorized
    404, # Not Found
    409, # Conflict
]
```

1. Limit Total Retries

```
# Bad: Retry forever
while True:
    try:
        return call()
    except:
        continue

# Good: Max attempts
for i in range(max_attempts):
    # ...
```

1. Exponential Backoff

```
Attempt 1: Immediate
Attempt 2: Wait 1s
Attempt 3: Wait 2s
Attempt 4: Wait 4s
```

1. Add Jitter

```
# Without jitter: All clients retry at same time
delay = 2 ** attempt

# With jitter: Clients spread out retries
delay = (2 ** attempt) + random.uniform(0, 1)
```

Production Scenario 15: Circuit Breaker State Machine

Circuit Breaker States:

```
CLOSED (Normal Operation)
  ↓ (failure rate > threshold)
OPEN (Failing Fast)
  ↓ (after timeout)
HALF-OPEN (Testing)
  ↓ (success → CLOSED, failure → OPEN)
```

Implementation:

```
from enum import Enum
from time import time

class CircuitState(Enum):
    CLOSED = "closed"
    OPEN = "open"
    HALF_OPEN = "half_open"

class CircuitBreaker:
    def __init__(self, failure_threshold=0.5, timeout=60, success_threshold=2):
        self.failure_threshold = failure_threshold
        self.timeout = timeout # seconds before trying half-open
        self.success_threshold = success_threshold # successes needed to close

        self.state = CircuitState.CLOSED
        self.failures = 0
        self.successes = 0
        self.total_requests = 0
        self.last_failure_time = None

    def call(self, func):
        if self.state == CircuitState.OPEN:
            if time() - self.last_failure_time > self.timeout:
```

```

        self.state = CircuitState.HALF_OPEN
        self.successes = 0
    else:
        raise CircuitBreakerOpen("Circuit breaker is open")

    try:
        result = func()
        self._on_success()
        return result
    except Exception as e:
        self._on_failure()
        raise

def _on_success(self):
    self.total_requests += 1
    if self.state == CircuitState.HALF_OPEN:
        self.successes += 1
        if self.successes >= self.success_threshold:
            self.state = CircuitState.CLOSED
            self.failures = 0
            self.total_requests = 0

def _on_failure(self):
    self.failures += 1
    self.total_requests += 1
    self.last_failure_time = time()

    if self.state == CircuitState.HALF_OPEN:
        self.state = CircuitState.OPEN
    elif self.state == CircuitState.CLOSED:
        if self.total_requests > 10: # Min requests before calculating rate
            failure_rate = self.failures / self.total_requests
            if failure_rate > self.failure_threshold:
                self.state = CircuitState.OPEN

```

Configuration Tuning:

```

# Aggressive (fail fast, recover quickly)
CircuitBreaker(
    failure_threshold=0.3, # 30% error rate
    timeout=30,           # Try recovery after 30s
    success_threshold=1   # 1 success to close
)

# Conservative (tolerate more failures, slow recovery)

```

```
CircuitBreaker(  
    failure_threshold=0.7,    # 70% error rate  
    timeout=120,              # Try recovery after 2min  
    success_threshold=5      # 5 successes to close  
)
```

From Hiring Manager Perspective

Interview Question 12:

"You implement retry logic with 3 attempts. Your service handles 1000 req/s with 10% error rate. What's the load after implementing retries?"

Correct Answer:

- Errors: 100 req/s
- Each error retries 3 times (1 initial + 2 retries)
- Extra load: $100 \times 2 = 200$ req/s
- Total: $1000 + 200 = 1200$ req/s (20% increase)
- At 20% error rate, would be: $1000 + (200 \times 2) = 1400$ req/s (40% increase)
- This is exponential, not linear

Interview Question 13:

"Your circuit breaker has 30s timeout. Service B is down for 60s. How many times does the circuit breaker try to close?"

Correct Answer:

- Circuit opens at $t=0$
- Tries half-open at $t=30$ (fails, reopens)
- Tries half-open at $t=60$ (succeeds, closes)
- Total: 2 attempts
- If B recovers at $t=45$, circuit stays open until $t=60$

Lab Exercises

Easy: Retry Amplification

Setup: Service A → Service B, B has configurable error rate

Exercise:

1. Baseline: Measure load on B with 10% error rate, no retries
2. Add retries (3 attempts) in A
3. Measure load increase on B
4. Calculate amplification factor
5. Increase error rate to 20%, 50%, measure again

Success Criteria:

- Understand retry amplification quantitatively
- See death spiral at high error rates

Moderate: Circuit Breaker Implementation

Setup: Implement circuit breaker from scratch

Exercise:

1. Implement state machine (CLOSED, OPEN, HALF-OPEN)
2. Configure: 50% failure threshold, 60s timeout
3. Test state transitions:
 - CLOSED → OPEN (inject 60% errors)
 - OPEN → HALF-OPEN (wait 60s)
 - HALF-OPEN → CLOSED (successful request)
 - HALF-OPEN → OPEN (failed request)

Success Criteria:

- All state transitions work correctly
- Circuit opens at threshold
- Fails fast when open

Hard: Retry Budget

Setup: Multi-service chain ($A \rightarrow B \rightarrow C$), each with retries

Exercise:

1. Set retry budget: Max 10% extra load allowed
2. Calculate: If A retries 3x and B retries 3x, what's the amplification?
3. Implement: Retry budget enforcement (reject retries when budget exhausted)
4. Test: Inject errors in C, verify retry budget prevents cascade

Success Criteria:

- Understand multiplicative amplification ($3 \times 3 = 9x$)
- Retry budget prevents cascade
- Some requests fail (budget exhausted) but system stable

Expert-Level Diagnostic Questions**Question 12:**

Service A calls B with 3-second timeout. B's P99 latency is 2.5s. What percentage of requests will timeout?

Expected Answer:

- P99 = 2.5s means 99% complete in < 2.5s
- 1% of requests take > 2.5s
- Some of these > 3s (timeout)
- Roughly 0.5% - 1% timeout
- This triggers retries, which amplifies load
- Better: Set timeout at P99.9 or P99.99

Question 13:

You call a payment API. Request times out. Should you retry? Why or why not?

Expected Answer:

- Depends on idempotency
 - If idempotent (idempotency key): Safe to retry
 - If not idempotent: May charge customer twice
 - Timeout doesn't mean request failed, just no response received
 - Best practice: Always implement idempotency for payments
-

[Continued in next file due to length...]